

Is there a tool with which we can check where these variables are stored? Yes, there are many. For example, the *objdump* tool can dump the whole executable file and show you the contents; but beginners would get overwhelmed by the details; so, a simpler tool will do. One such simple tool is *nm*.

## Using the *nm* tool

The *nm* manpage says that it's a tool to "...list symbols from object files". So how does one use *nm*?

First, assume that we stored the program in */tmp/allocation.c* file. Now, compile it and create an executable, as shown below:

```
ganesh@linux-2rc:~$ cc -std=c99 /tmp/allocation.c
```

(Since I use some C99 features like single line comments, I compiled the program in C99 mode.)

Now, type *nm ./a.out* (even just typing *nm* will do—if there are no arguments given to *nm*, it assumes that it should take the input as *a.out*) and you'll get some cryptic output as follows:

```
ganesh@linux-2rc:~$ nm ./a.out
08049650 B bss1
08049648 b bss2
08049654 B bss3
08049638 A __bss_start
08048374 t call_gmon_start
08049638 b completed.5764
...
```

I haven't shown the whole output since it will fill a whole page. Where are the symbols that we did not type in the program, coming from? They have been inserted behind the scenes by the compiler for various reasons. We can ignore them for now.

Now, what are those strange numbers, followed by the letters (such as 'b', 'B', 't')? The numbers are the symbol's value, followed by the symbol type (displayed as a letter) and the symbol name.

The symbol type requires more explanation. A lowercase symbol means that it is local (to the file) and an uppercase letter means that it is global (externally available from the file). Here are the symbol types and their meanings that are of interest to us:

"B" The symbol is in the uninitialised data section (known as BSS).  
 "D" The symbol is in the initialised data section.  
 "T" The symbol is in the text (code) section.  
 "U" The symbol is undefined.

Oh good, that's a short list. Now, let's look out for the symbols that are relevant to us (by redirecting the output to *grep* command), and discuss them in detail:

```
ganesh@linux-2rc:~$ nm ./a.out | grep bss
08049650 B bss1
08049648 b bss2
```

```
08049654 B bss3
```

Variables *bss1* and *bss3* got allocated in the BSS section (global). Since we put the *storage* class as static for the variable *bss2*, it is listed as 'b' (lower case 'b' means that it is accessible only within that file) and is also allocated in the BSS section:

```
ganesh@linux-2rc:~$ nm ./a.out | grep init
08049628 D init1
0804962c D init2
08049630 D init3
```

No surprises here for variables *init1*, *init2* and *init3*: since they are explicitly initialised, they got allocated into the initialised data section.

```
ganesh@linux-2rc:~$ nm ./a.out | grep local
08049640 b local3.1847
08049634 d local4.1848
```

Only the *local3* and *local4* are allocated global memory. Since *local3* is uninitialised, it is allocated in BSS; and since *local4* is explicitly initialised, it is allocated in the initialised data segment. As both are local (to the function), they are indicated by smaller case letters ('b' and 'd', respectively). Why are the names suffixed with some numbers here? Presumably, since they are local to the function and to avoid accidental mixing them up with other local variables with the same name, they have been suffixed by some numbers. (**Note:** Compilers differ in their approaches in treating local static variables; this approach is for GCC.)

In the output, the following few symbols are also of interest to us:

```
080483f4 T main
U malloc@@GLIBC_2.0
U printf@@GLIBC_2.0
```

The *main* function is allocated in the text segment; obviously, we can access this function from outside the file (to start the execution). So, the type of this symbol is 'T'.

The *malloc* and *printf* functions used in the program are not defined in the program itself (the header files only *declare* them, they don't *define* them); they are defined in the shared library GLIBC, version 2.0—that's what the suffix "@@GLIBC\_2.0" implies.

Hopefully, this article has demystified some of the behaviour of natively executable programs. You can take this as a starting point and explore more by yourself. Read about ELF and COFF file formats, about how segments other than the ones I've described here, are useful, etc. Check the GCC manual for more details. 

### About the author:

**S G Ganesh** is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at [sgganesh@gmail.com](mailto:sgganesh@gmail.com).